



Team Project 1 - CAP

Html Fundamental



What is HTML?

- HTML stands for HyperText Markup Language.
- HTML is a **Markup** language.
- It is the standard language used to **create and structure** content on the web.
- HTML provides the structure, while CSS provides style and JavaScript provides interactivity.
- Every web page you visit is built with HTML.

Its primary purpose is to annotate text so that a web browser knows how to structure and display content, such as identifying what is a heading, a paragraph, or a hyperlink.

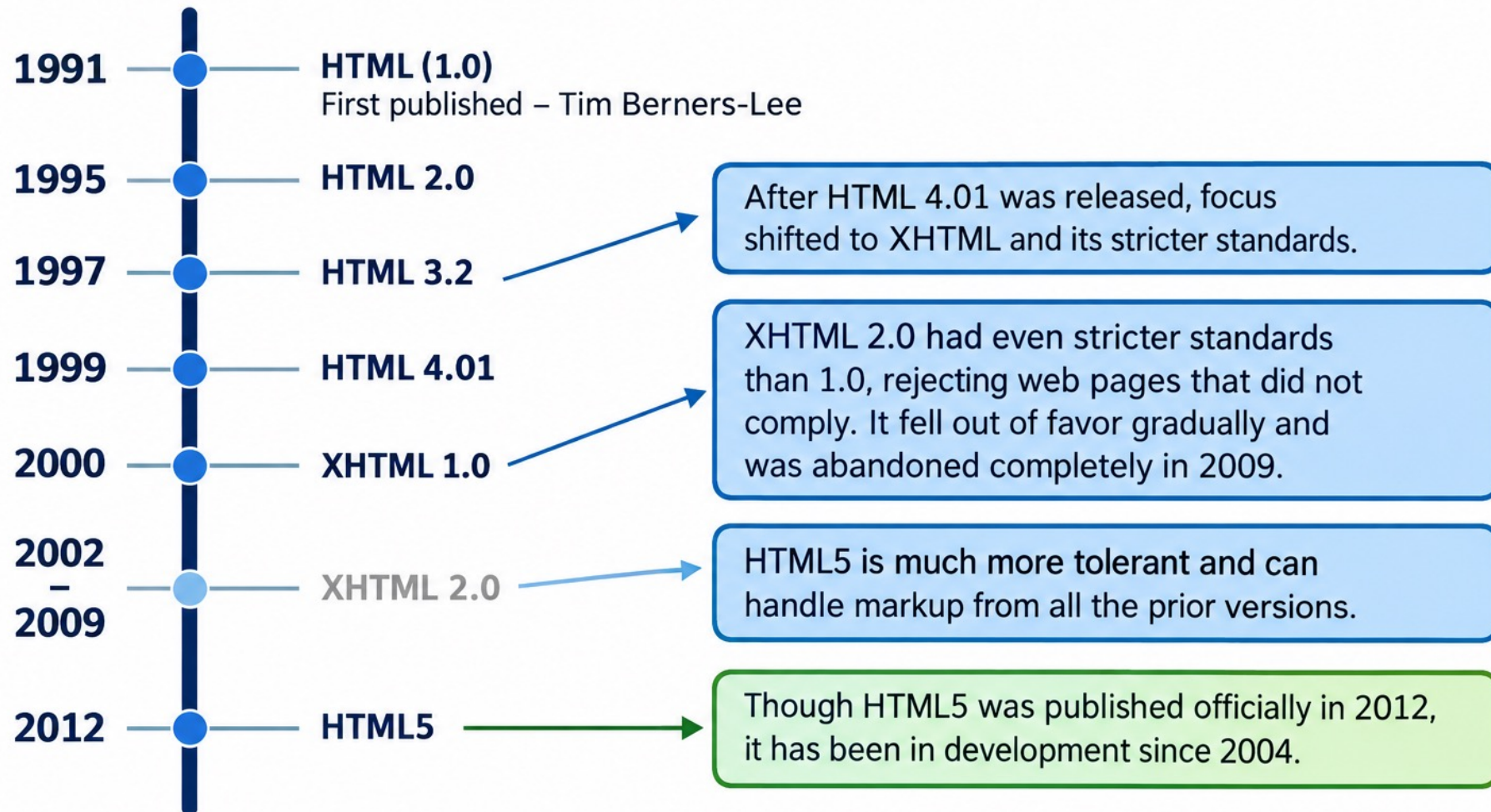


Differences between Markup & Programming

Feature	Markup Language (HTML)	Programming / Scripting Language (JavaScript, Python)
Primary Purpose	Defines the structure and visual layout of text and media.	Controls the behavior , logic, and functionality of an application.
Logic & Control Flow	No if/else statements, loops, or complex conditional logic.	Built entirely around conditional logic, functions, and algorithms.
Data Handling	Cannot create variables, manipulate data, or perform math (e.g., it can't calculate $1 + 1$).	Easily stores, retrieves, alters, and calculates complex data.
Analogy	Like a blueprint or the skeleton of a building.	Like the electrical or plumbing systems that make the building functional.

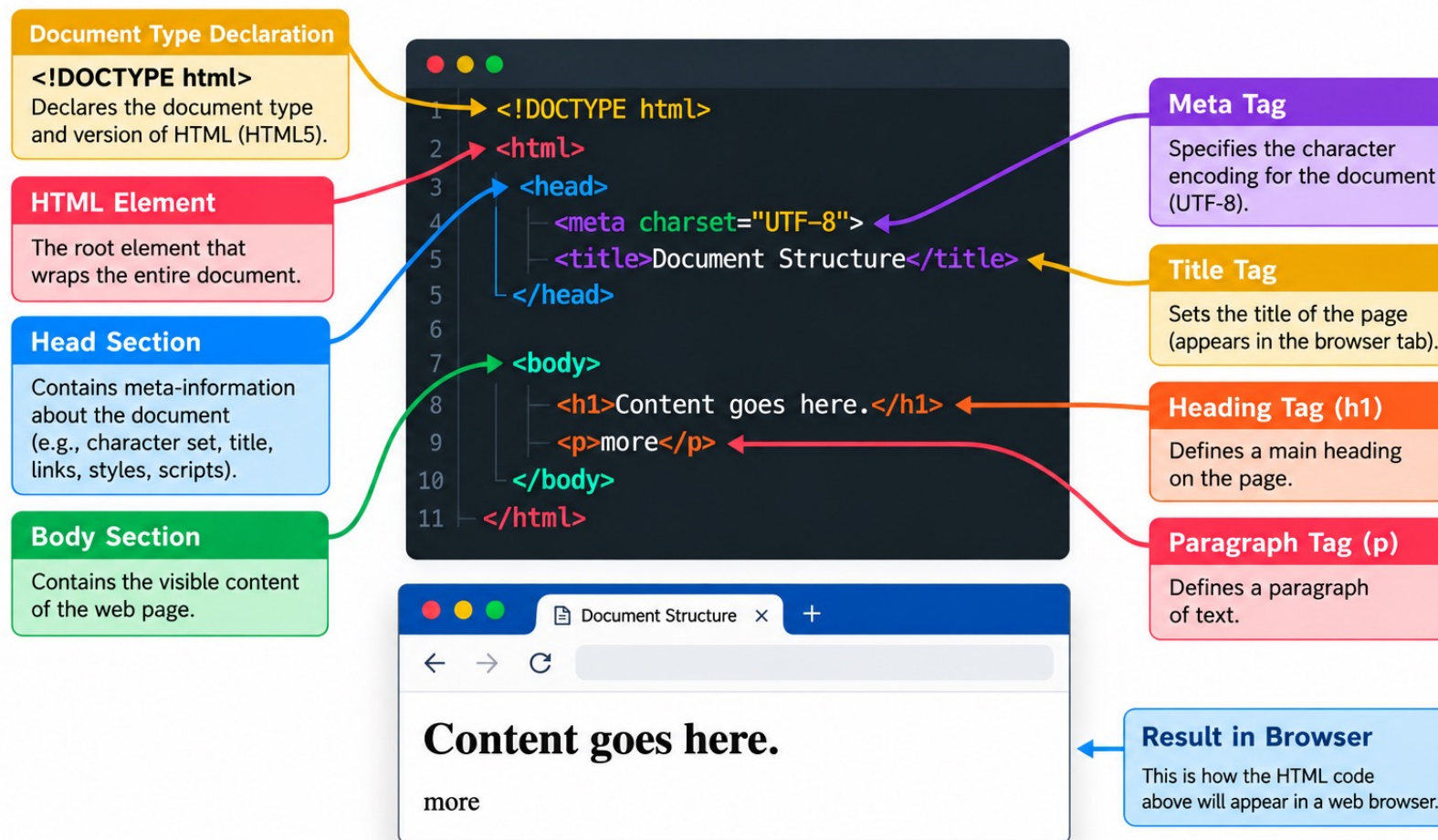


History HTML?





Anatomy of HTML Document





HTML Tags & Elements

- A tag is a special HTML keyword written inside `< >` that tells the browser what something is.
Example: `<html>`, `<p>`, `<table>`
- **Element** = Opening Tag + Content + Closing Tag
Example: `<p>This is a paragraph.</p>`
- **Attributes** provide additional information about an element and are placed inside the **opening tag**.

```
<p>This is a paragraph.</p>
```

```

```

```
<a href="https://example.com" target="_blank">
```

```
    Visit Website
```

```
</a>
```



Self-closing / Void HTML Elements

Some HTML elements **do not have a closing tag** because they cannot contain content. These are officially called **void elements**.

Common Void Elements:

- **** - displays an image
- **<input>** - creates a form input
- **
** - inserts a line break
- **<hr>** - creates a thematic break
- **<meta>** - defines document metadata
- **<link>** - links external resources
- **<source>** - specifies media sources

Important Note

In **HTML5**, the **/** is **not required**

```



<input type="text" name="username">
<input type="text" name="username" />

<br>
<br />

<hr>
<hr />

<meta charset="UTF-8">
<meta charset="UTF-8" />

<link rel="stylesheet" href="style.css">
<link rel="stylesheet" href="style.css" />
```



<head> & <body>

<head> **Metadata/Information about the page**

Contains information **about the document** that is generally **not displayed as page content**.

Common elements:

- <title> — browser tab title
- <meta> — metadata, character encoding, viewport
- <link> — external resources such as CSS
- <style> — internal CSS
- <script> — JavaScript

<body> **Content of the Page**

Contains the **content displayed to the user** in the browser.

Examples:

- Headings
- Paragraphs
- Images
- Links
- Tables, forms, lists, etc.

<head> = information ABOUT the page
<body> = content INSIDE the page



<head> & <body>

```
<head>
  <meta charset="UTF-8">
  <title>My Website</title>
  <link rel="stylesheet" href="style.css">
</head>

<body>
  <h1>Welcome to My Website</h1>
  <p>This is a paragraph.</p>
  
</body>
```



HTML Text Tags & Formatting

Tag	Description	Example Code	Result
<code><p></code>	Defines a paragraph	<code><p>This is a paragraph.</p></code>	This is a paragraph.
<code><h1></code>	Defines a level 1 heading	<code><h1>This is a heading 1</h1></code>	This is a heading 1
<code><h2></code>	Defines a level 2 heading	<code><h2>This is a heading 2</h2></code>	This is a heading 2
<code><h3></code>	Defines a level 3 heading	<code><h3>This is a heading 3</h3></code>	This is a heading 3
<code><h4></code>	Defines a level 4 heading	<code><h4>This is a heading 4</h4></code>	This is a heading 4
<code><h5></code>	Defines a level 5 heading	<code><h5>This is a heading 5</h5></code>	This is a heading 5
<code><h6></code>	Defines a level 6 heading	<code><h6>This is a heading 6</h6></code>	This is a heading 6
<code>
</code>	Inserts a line break	Line 1 <code>
</code> Line 2	Line 1 Line 2
<code><hr></code>	Defines a thematic break (horizontal rule)	<code><hr></code>	
<code><blockquote></code>	Defines a block of quoted text	<code><blockquote>This is a quote.</blockquote></code>	This is a quote.
<code><pre></code>	Defines preformatted text (preserves spaces and line breaks)	<code><pre>Line 1 Line 2</pre></code>	Line 1 Line 2
<code><code></code>	Defines a piece of computer code	<code><code>console.log("Hello");</code></code>	console.log("Hello");



HTML Text Tags & Formatting

Tag	Description	Example Code	Result
	Defines bold text	<code>This is bold text</code>	This is bold text
	Defines emphasized text	<code>This is emphasized text</code>	<i>This is emphasized text</i>
<i>	Defines a part of text in an alternate voice or mood	<code><i>This is italic text</i></code>	<i>This is italic text</i>
<small>	Defines smaller text	<code><small>This is smaller text</small></code>	This is smaller text
	Defines important text	<code>This is important text</code>	This is important text
<sub>	Defines subscripted text	<code><sub>This is subscripted text</sub></code>	This is _{subscripted} text
<sup>	Defines superscripted text	<code><sup>This is superscripted text</sup></code>	This is ^{superscripted} text
<ins>	Defines inserted text	<code><ins>This is inserted text</ins></code>	<u>This is inserted text</u>
	Defines deleted text	<code>This is deleted text</code>	This is deleted text
<mark>	Defines marked/highlighted text	<code><mark>This is marked text</mark></code>	This is marked text



HTML Lists

- **Ordered List** `<ol>` Creates a **numbered list**.
- **Unordered List** `<ul>` Creates a **bulleted list**.
- **List Item** `<li>` Defines each **item inside** an `<ol>` or `<ul>` list.

List Type	HTML Code	Result
Ordered List <code></code> Creates a numbered list.	<pre> Bread Milk </pre>	1. Bread 2. Milk
Unordered List <code></code> Creates a bulleted list.	<pre> Bread Milk </pre>	• Bread • Milk



Hyperlinks

- Hyperlinks connect one page to **another page** or an **external website**.
- The `<a>` tag defines a hyperlink.

Common Attributes

- **href** - specifies the destination URL.
- **target="_self"** - opens the link in the **same tab** (default).
- **target="_blank"** - opens the link in a **new tab**.

Link to a Section on the Same Page,
Use an element's **id** as an anchor

```
<a href="https://example.com">Visit Website</a>

<a href="https://example.com" target="_blank">
|   Visit Website
</a>

<a href="#contact">Go to Contact</a>

<h2 id="contact">Contact Us</h2>
```



Images & Media

HTML allows you to add **images**, **audio**, and **video** to a web page.

Images - `<img>`

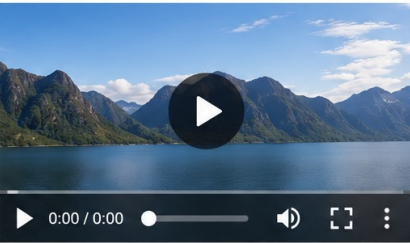
- Embeds an image in the page.
- **src** - path or URL of the image.
- **alt** - alternative text if the image cannot be displayed.
- **width / height** - control image dimensions.

Audio - `<audio>`

- Embeds audio content.
- **controls** - displays play, pause, volume, etc.
- `<source>` specifies the audio file and its type.

Video - `<video>`

- Embeds video content.
- **controls** - displays video controls.
- **width / height** - control video dimensions.
- `<source>` specifies the video file and its type.

HTML Code	Result
<pre></pre>	
<pre><audio controls> <source src="song.mp3" type="audio/mpeg"> </audio></pre>	
<pre><video width="400" controls> <source src="movie.mp4" type="video/mp4"> </video></pre>	



Tables

HTML tables organize data into **rows** and **columns**.

Main Table Tags:

- **<table>** - defines the entire table.
- **<tr>** - defines a table row.
- **<th>** - defines a header cell.
- **<td>** - defines a data cell.

Extra Table Tags:

- **<thead>** - groups the **header** rows.
- **<tbody>** - groups the **main data** rows.
- **<tfoot>** - groups the **footer/summary** rows.

HTML Code

```
<table>
  <thead>
    <tr>
      <th>Name</th>
      <th>Age</th>
      <th>Country</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>Ahmad</td>
      <td>25</td>
      <td>Palestine</td>
    </tr>
    <tr>
      <td>Maria</td>
      <td>30</td>
      <td>Italy</td>
    </tr>
  </tbody>
</table>
```

Result

Name	Age	Country
Ahmad	25	Palestine
Maria	30	Italy



HTML Comments

- Comments are **not displayed** in the browser.
- Used to **explain code** or leave notes for developers.
- Helpful for **documentation, debugging, or reminders**.
- Comments can hide code temporarily.

HTML Code	Result
<pre><!-- This is a single-line comment --> <p>This is visible text.</p> <!-- This is a multi-line comment It can span multiple lines --> <!-- <p>This paragraph is hidden because it is inside a comment</p> --></pre>	This is visible text.



Form & Inputs

HTML forms are used to **collect user input** and send it for processing.

<form> Tag

- **action** - specifies the URL where the form data is sent.
- **method** - specifies how the data is sent:
 - GET - sends data in the URL.
 - POST - sends data in the request body.

Common Input Types

- **text** - single-line text.
- **email** - email address.
- **password** - hidden password input.
- **number** - numeric input.
- **date** - date selector.
- **checkbox** - select zero or more options.
- **radio** - select one option from a group.
- **file** - select a file.
- **submit** - submit the form.



Form & Inputs

HTML Code

```
<form action="/register" method="post">
  <label>Name:</label>
  <input type="text" name="name">

  <label>Email:</label>
  <input type="email" name="email">

  <label>Gender:</label>
  <input type="radio" name="gender" value="male">
  Male
  <input type="radio" name="gender" value="female">
  Female

  <label>
    <input type="checkbox" name="terms">
    Accept Terms
  </label>

  <input type="submit" value="Register">
</form>
```

Result

Name:

Email:

Gender:

☐ Male ☐ Female

☐ Accept Terms

Register



Form & Inputs

HTML Code	Result
<code><input type="button"></code>	Button
<code><input type="checkbox"></code>	☐ ☒ ☐
<code><input type="color"></code>	<input type="color" value="red"/>
<code><input type="date"></code>	<input type="date" value="dd-mm-yyyy"/>
<code><input type="email"></code>	<input type="email" value="farazc60@gmail.com"/>
<code><input type="file"></code>	<input type="file" value="Choose File"/> No file chosen
<code><input type="hidden"></code>	
<code><input type="image"></code>	<input type="image" value="image"/>
<code><input type="number"></code>	<input type="number" value="5"/>
<code><input type="password"></code>	
<code><input type="radio"></code>	☒ ☐
<code><input type="range"></code>	<input type="range" value="50"/>
<code><input type="reset"></code>	<input type="reset" value="Reset"/>
<code><input type="submit"></code>	<input type="submit" value="Submit"/>
<code><input type="text"></code>	codewithfaraz
<code><input type="url"></code>	<input type="url" value="https://www.codewithfaraz.com"/>



Forms & Inputs

- **name** is important — it identifies the input when form data is submitted.
- **value** specifies the value sent to the server.
- Use **<label>** with **for** and **id** to associate text with an input and improve accessibility.
- Useful validation attributes include: **required**, **min**, **max**, **minlength**, **maxlength**, and **pattern**.
- Radio buttons with the **same name** belong to the same group, allowing only one to be selected.
- **placeholder** provides a hint inside an input, but it should **not replace a <label>**.
- **disabled** inputs cannot be edited **and are not submitted**;
- **readonly** inputs cannot be edited but their values **are submitted**.

Note that to submit the user input you need to add a submit button inside the form.

- See this link to learn about form elements: https://www.w3schools.com/html/html_form_elements.asp
- See this link to learn about input types: https://www.w3schools.com/html/html_form_input_types.asp

HTML validation improves the user experience, but server-side validation is still required.





Buttons

- The **<button>** tag defines a **clickable button** in an HTML document.
- **<button>** has an **opening and closing tag**, so it can contain text and other HTML elements such as ****, ****, and **<i>**.
- **The type Attribute**, The **type** attribute defines the button's behavior:
 - **type="button"** - a regular button with no default form action.
 - **type="submit"** - submits the form data.
 - **type="reset"** - resets the form fields to their initial values.

HTML Code	Result
<pre><button>Click Me</button></pre>	
<pre><button> Save </button></pre>	

Important: Inside a **<form>**, a **<button>** without a type defaults to **type="submit"**.



Labels

- The tag is used when you have a form or input elements in which, the label is associated with an input element via the “for” attribute that the input id.
- The element does not render as anything special for the user. However, it provides a usability improvement for mouse users, because if the user clicks on the text within the element, it toggles the control associated with it.
- The “for” attribute of the tag should be equal to the id attribute of the related element to bind them together.

HTML Code	Result
<pre><label for="username">Username:</label> <input type="text" id="username" name="username"></pre>	Username:



Semantic HTML

Semantic HTML uses tags that **describe the meaning and purpose of the content**, not just how it looks.

Why Use Semantic HTML?

- **Meaningful Structure** — clearly describes the role of each part of the page.
- **SEO** — helps search engines better understand and index page content.
- **Accessibility** — helps screen readers and other assistive technologies understand page structure and navigation.
- **Code Readability** — makes HTML easier for **developers and LLMs/AI tools** to understand and work with.

Common Semantic Tags

- **<header>** — introductory/header content.
- **<nav>** — navigation links.
- **<main>** — main content of the page.
- **<section>** — thematic section of content.
- **<article>** — self-contained content, such as a post or article.
- **<aside>** — related or supplementary content.
- **<footer>** — footer information.

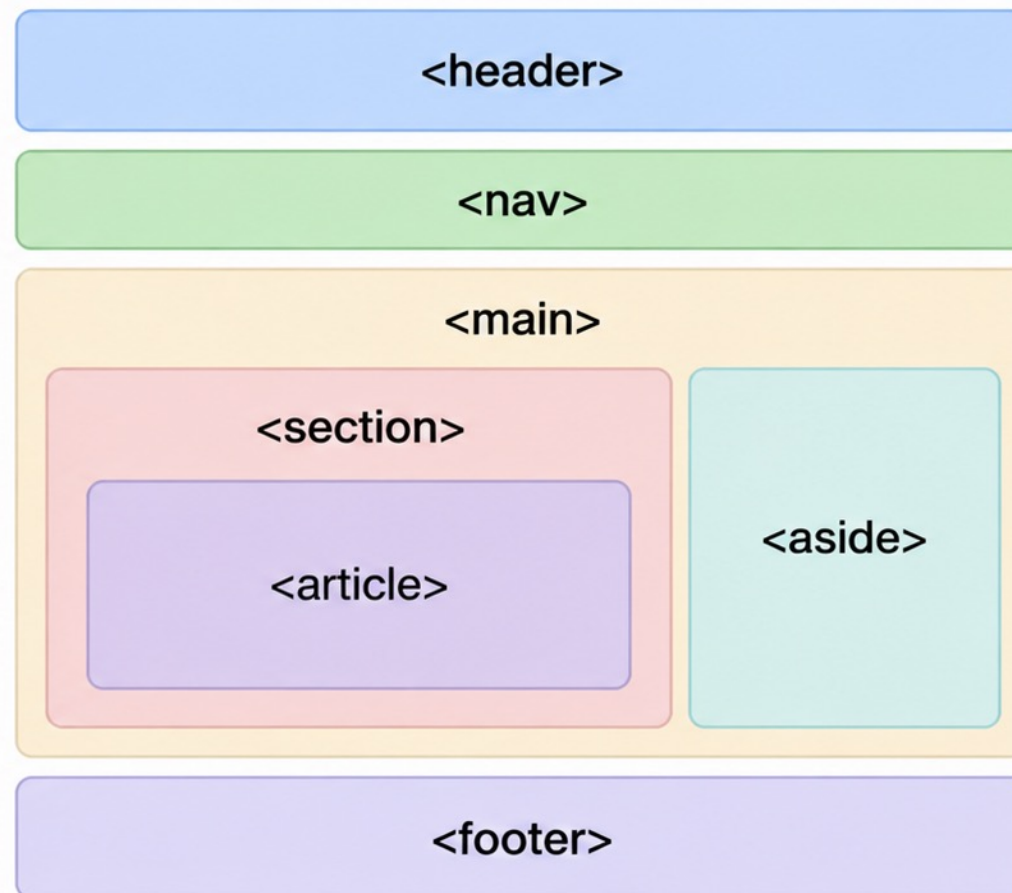


Semantic HTML

HTML Code

```
<header>...</header>
<nav>...</nav>
<main>
  <section>
    <article>...</article>
  </section>
  <aside>...</aside>
</main>
<footer>...</footer>
```

Page Structure (Visual)





Block-Level vs. Inline HTML Elements

HTML elements are commonly displayed as either **block-level** or **inline** elements.

Block-Level Elements

- Start on a **new line**.
- By default, take up the **full available width** of their container.
- The next element normally starts on a **new line**.

Examples: `<p>` · `<div>` · `<h1>`—`<h6>` · `<ul>` · `<ol>` · `<li>` · `<table>`

Inline Elements

- Do **not** start on a new line.
- Take up only as much width as their **content requires**.
- Can appear next to other inline content on the same line.

Examples: `<a>` · `<span>` · `<img>`



Block-Level vs. Inline HTML Elements

HTML Code

```
<h2>First Heading</h2>
```

```
<p>This is a paragraph.</p>
```

```
<div>This is a div.</div>
```

```
<span>Hello</span>
```

```
<a href="#">Visit Website</a>
```

```

```

Result (with borders)

Block-Level Elements

First Heading

This is a paragraph.

This is a div.

Each element
takes full width
and starts on a
new line

Inline Elements

Hello

[Visit Website](#)



Elements take only
as much width as
necessary and stay
on the same line



Block-level element (border)



Inline element (border)



HTML Best Practices

- **Use Semantic HTML**

Prefer meaningful tags such as `<header>`, `<nav>`, `<main>`, `<section>`, and `<footer>`.

- **Use Proper Document Structure**

Include `<!DOCTYPE html>`, `<html>`, `<head>`, and `<body>`.

- **Close Elements Correctly**

Make sure elements that require closing tags are properly closed and nested.

- **Use Consistent Indentation**

Keep code clean, organized, and easy to read.

- **Use Meaningful Names**

Choose clear and descriptive id, class, and name values.

- **Use Accessibility Features**

Use `<label>` for form fields, `alt` for images, and appropriate semantic elements.

- **Separate HTML, CSS & JavaScript**

HTML → **Structure**

CSS → **Presentation / Style**

JavaScript → **Behavior / Interaction**

- **Avoid Deprecated HTML**

Use modern HTML elements and CSS instead of obsolete tags and attributes.

- **Validate Your HTML**

Check your markup using the official [W3C Markup Validation Service](https://validator.w3.org/).

- **Keep It Simple & Readable**

Write HTML that is easy for **humans, browsers, search engines, and AI tools** to understand.

Author

Amjad Ataallah

Email: amjad.atallah@najah.edu
Linkedin: <https://www.linkedin.com/in/ataallah/>
Website: <https://amjadataallah.github.io>



Yanal Sweseh

Email: yanal.swaseh@najah.edu
Linkedin: <https://www.linkedin.com/in/yanalsweseh/>
Website: <https://github.com/yswesieh>

